



EVENTS — dis-bot v2.0 Stable

Документ фиксирует **контракт событий** (Event Bus) для билда **dis-bot v2.0**.



В v2.0 события и их payload считаются **частью публичного API проекта**. - Запрещено менять имена событий. - Запрещено менять смысл/типы полей payload. - Разрешено только **добавлять новые события** или **добавлять новые поля с backward compatibility**.

Что такое Event Bus

Event Bus — это слой слабой связанности между модулями:

- Producers (источники) **emit** события
- Consumers (подписчики) **subscribe** и реагируют

Файл: `utils/events_bus.py`

Базовые правила

- Все обработчики вызываются **последовательно**.
- Ошибка в обработчике **не валит процесс**.
- Событие не должно зависеть от UI.



Список событий v2.0

Event	Назначение	Producers	Consumers
<code>daily_claimed</code>	пользователь забрал дэйлик	<code>fun_slesh/daily.py</code>	<code>fun_slesh/achievements_engine.py</code>
<code>game_win</code>	пользователь победил в игре	<code>fun_slesh/games.py</code>	<code>fun_slesh/achievements_engine.py</code>
<code>rep_given</code>	пользователю выдали репутацию	<code>fun_slesh/rep_and_mood.py</code>	<code>fun_slesh/achievements_engine.py</code>



Контракты событий (payload)

1) `daily_claimed`

Когда эмитится: после успешного получения награды `/дэйлик`.

Producer: `fun_slesh/daily.py`

Emit:

```
await emit("daily_claimed", user_id=..., streak=..., amount=...)
```

Payload (обязательные поля): - `user_id: int` — кто получил дэйлик - `streak: int` — текущая серия после начисления - `amount: int` — сколько монет начислено

Семантика: - событие означает, что **баланс уже начислен** в экономику; - обработчики не должны повторно начислять монеты.

Основные потребители: - Achievements Engine: выдаёт `first_daily`, `streak_3/7/14/30`.

2) `game_win`

Когда эмитится: после победы в PvE или PvP игре (если предусмотрено награждение/победа).

Producer: `fun_slesh/games.py`

Emit:

```
await emit("game_win", user_id=..., game=...)
```

Payload (обязательные поля): - `user_id: int` — кто победил - `game: str` — код игры

Возможные значения `game` в v2.0: - `"rps"` — КНБ (PvE и PvP) - `"guess"` — угадай число

Семантика: - событие означает «победа зафиксирована»; - награждение монетами может быть уже выполнено producer'ом.

Основные потребители: - Achievements Engine: выдаёт `first_win_rps` / `first_win_guess`.

3) `rep_given`

Когда эмитится: после успешной выдачи репутации командой `/репа`.

Producer: `fun_slesh/rep_and_mood.py`

Emit:

```
await emit("rep_given", user_id=..., given_by=..., date=...)
```

Payload (обязательные поля): - `user_id: int` — кому дали репутацию - `given_by: int` — кто дал репутацию - `date: str` — дата выдачи в формате `YYYY-MM-DD` (MSK-логика)

Семантика: - событие означает, что запись репутации уже сохранена; - для ачивок важно поле `given_by`.

Основные потребители: - Achievements Engine: выдаёт ачивку `first_rep_given` тому, кто выдал.

Версионирование событий

Разрешено (backward compatible)

- добавлять новые события;
- добавлять новые поля payload, если старые consumers могут их игнорировать.

Запрещено

- переименовывать событие;
 - менять типы полей;
 - менять смысл полей (например, `amount` в `daily_claimed` должен оставаться «начислено монет»).
-

Рекомендации для будущих расширений (поверх v2.0)

Если добавляется новая функциональность (ML, интеграции, новые игры):

1) **не модифицировать** существующие события 2) добавлять новые события, например: - `etl_completed` - `model_trained` - `shop_updated` - `match_result`

3) описывать их в `EVENTS.md` с: - producer - consumer - payload - семантика

`EVENTS.md` фиксирует контракт событий `dis-bot v2.0 Stable`.